Descrizione del progetto finale

Nome del progetto di laboratorio:

For You

Gruppo

Informazioni sul gruppo di laboratorio.

Nome del gruppo:	La Rosa
Componenti:	Salvatore La Rosa
Github/Gitlab Repo Link:	• Github id: *Salvatore-LaRosa* • https://github.com/Salvatore-LaRosa/ForYou2

Date

Le date principali del documento.

Data di sottomissione della proposta di progetto	28/04/2025
Data di accettazione della proposta di progetto	07/05/2025

Descrizione breve

Descrizione della App in una frase.

For You è un'app Android sviluppata in Kotlin, che consente agli utenti di gestire l'assunzione dei farmaci in modo efficiente e sicuro, attraverso un'interfaccia sviluppata con Jetpack Compose, un database locale Room e un sistema di notifiche automatiche basato su BroadcastReceiver e AlarmManager.

Definizioni

Di seguito la definizione dei termini, abbreviazioni e acronimi utilizzati.

Termine	Definizione

1. Contesto del progetto

Dipartimento di Informatica, C.so Svizzera 185. Tel. 0116706711 Fax. 011751603

Lunghezza suggerita: 2 pagine

1.1. Situazione attuale

Attualmente sul Play Store esistono molte app che ricordano agli utenti di assumere i farmaci (es. Medisafe, MyTherapy). Tuttavia, queste app presentano diverse carenze:

- Poca intuitività per utenti anziani
- Report poco dettagliati

1.2. Benefici e creazione di valore

L'app Habit Tracker offrirà un'esperienza semplice e intuitiva, con particolare attenzione alla sicurezza dei dati tramite autenticazione biometrica e un'interfaccia personalizzabile. La possibilità di visualizzare resoconti dettagliati mensili permetterà agli utenti di monitorare e migliorare il proprio comportamento.
La nostra app introduce:

- Report dettagliati riguardo le assunzioni dimenticate/puntuali
- Interfaccia semplice e adatta anche ad utenti non esperti
- Personalizzazione elevata delle terapie e dei promemoria

1.3. Obiettivi del progetto

Elenco di aspetti specifici e verificabili che realizzerete nella vostra App e che la renderanno "appetibile".

Interfaccia intuitiva basata su Jetpack Compose	L'app offrirà una UI pulita e interattiva, sviluppata con Jetpack Compose per garantire flessibilità e facilità di aggiornamento. Gli utenti potranno aggiungere, modificare e visualizzare i loro farmaci da assumere giornalmente, settimanalmente e mensilmente, con un'interfaccia che riduce il rischio di errori e semplifica la gestione dello stato.
Gestione dei dati locali tramite Room	I dati relativi alle assunzioni saranno memorizzati localmente utilizzando la libreria Room che permetterà di gestire le abitudini e i progressi dell'utente in modo persistente e sicuro, anche offline.

Dipartimento di Informatica, C.so Svizzera 185. Tel. 0116706711 Fax. 011751603

Implementazione dell'architettura MVVM	L'intera app sarà sviluppata seguendo l'architettura MVVM (Model-View-ViewModel), che separa chiaramente la logica di business (Model) dalla UI (View). Il ViewModel fungerà da intermediario, mantenendo i dati aggiornati e garantendo che la UI sia sempre reattiva. Grazie a questa separazione, la manutenzione dell'app sarà semplificata e verrà garantita una maggiore facilità di test.
Uso di LiveData per l'aggiornamento dinamico delle interfacce	La comunicazione tra il ViewModel e le interfacce utente sarà gestita tramite LiveData, che permette alle attività e ai fragment di osservare i cambiamenti dei dati in modo automatico. Questo approccio renderà l'interfaccia utente sempre aggiornata senza dover gestire manualmente le notifiche di aggiornamento.

Dipartimento di Informatica, C.so Svizzera 185. Tel. 0116706711 Fax. 011751603

2. PROFILO DEL PROGETTO

Lunghezza suggerita: 1 pagina

2.1. Ambito del progetto

For You si propone di realizzare un sistema completo per la gestione locale delle assunzioni di farmaci, senza dipendere da servizi remoti.

L'app sarà in grado di:

- Registrare utenti con credenziali salvate localmente.
- Memorizzare farmaci e relativi orari.
- Programmare notifiche automatiche per ricordare le assunzioni.
- Visualizzare report periodici delle assunzioni.

Alla fine del progetto, l'app sarà completamente funzionale sul dispositivo locale. Un'eventuale estensione futura potrebbe includere sincronizzazione cloud o esportazione dei dati per i medici.

2.2. Profilo della soluzione

L'app è costruita su architettura **MVVM**, garantendo separazione dei livelli logici e facile manutenibilità.
L'interfaccia è sviluppata con **Jetpack Compose**, mentre i dati sono gestiti da **Room** con flussi **Kotlin Flow** e operazioni asincrone tramite **Coroutines**.
Le notifiche sono gestite da un **BroadcastReceiver** dedicato, ReminderReceiver, che genera notifiche locali basate sui farmaci pianificati.

Dipartimento di Informatica, C.so Svizzera 185. Tel. 0116706711 Fax. 011751603

3. Vincoli e assunti (Requirements)

Lunghezza suggerita: 2 pagine

3.1. Vincoli tecnologici

Functional Requirements

- **Autenticazione locale** con email e password, supportata da **PIN** e **autenticazione biometrica** tramite API BiometricPrompt.
- **Gestione dei farmaci** con possibilità di aggiungere, modificare e rimuovere elementi.
- **Persistenza dati locale** tramite **Room Database**.
- **Visualizzazione dinamica** dei farmaci giornalieri attraverso liste aggiornate in tempo reale con **Flow** e **LiveData**.
- **Programmazione di notifiche automatiche**, che avvisano l'utente all'orario specificato per ciascun farmaco.
- **Visualizzazione di resoconti mensili** con grafici e dati aggregati.

Non-functional Requirements

- **Reattività**: tutte le operazioni di I/O e database sono gestite tramite **Kotlin Coroutines** per evitare blocchi della UI.
- **Compatibilità**: supporto garantito per dispositivi Android 8.0 (API 26) e successivi.
- **Sicurezza**: utilizzo di EncryptedSharedPreferences per salvare dati sensibili (email utente, preferenze biometriche).
- **Scalabilità e manutenibilità**: ottenute grazie al pattern architetturale MVVM.

Il progetto sarà sviluppato esclusivamente per la piattaforma Android, utilizzando il linguaggio di programmazione Kotlin. L'app è sviluppata per versioni di Android 8.0 (Oreo) e superiori, garantendo compatibilità con la maggior parte dei dispositivi Android in uso oggi.

3.2. Eventuali assunti

- L'app opera completamente in locale; nessuna connessione a servizi cloud è necessaria.
- Le notifiche sono gestite tramite AlarmManager e BroadcastReceiver.
- Ogni utente ha un database locale isolato per la propria sessione.

Dipartimento di Informatica, C.so Svizzera 185. Tel. 0116706711 Fax. 011751603

4. Design

Lunghezza suggerita: 2-3 pagine

3.1. Struttura architetturale

L'architettura dell'app segue il pattern **MVVM (Model–View–ViewModel)**, integrando componenti **Android Jetpack** (Compose, Room, LiveData, ViewModel, Coroutines) per garantire modularità, testabilità e robustezza.

Struttura del progetto:

- data/
  - local/
    - dao/: Interfacce DAO per l'accesso ai dati Room (MedicineDAO, UserDAO)
    - entity/: Entità persistenti (User, Medicine, MedicineIntake)
    - **UserDatabase**: Implementazione del database Room
  - session/
    - **SessionManager**: Gestisce la sessione dell'utente memorizzando e recuperando i dati della sessione da SharedPreferences.
- navigation/
  - **BottonNavGraph**: Definisce il grafico di navigazione visualizzabile dalla barra inferiore (Home, Report, Settings)
  - **NavGraph**: Definisce il grafico di navigazione principale dell'applicazione. Questo grafico include il flusso di autenticazione iniziale (accesso, registrazione, impostazione del PIN)
- notification/
  - **ReminderReceiver**: Gestione dei promemoria e notifiche
- repository/: Layer di astrazione per l'accesso ai dati
- ui/
  - screens/
    - **AddMedicineScreen**: Schermata per l'aggiunta di un nuovo medicinale. L'utente può inserire nome, frequenza, quantità, data di fine, ora e note. Include anche la logica per richiedere autorizzazioni di notifica e programmare promemoria.
    - **HomeScreen**: La schermata principale dell'applicazione che mostra l'elenco dei medicinali per la data selezionata. L'utente può selezionare una data diversa, aggiungere un nuovo medicinale e visualizzarne i dettagli.
    - **LoginScreen**: Schermata per eseguire il login tramite email e password
    - **PinLoginScreen**: Una schermata per l'accesso tramite PIN, che offre anche l'autenticazione biometrica come alternativa. Gestisce l'inserimento del PIN, la convalida e le richieste biometriche.
    - **RegisterScreen**: Schermata per registrare email e password

Dipartimento di Informatica, C.so Svizzera 185. Tel. 0116706711 Fax. 011751603

        - **ReportScreen**: Schermata che visualizza un resoconto mensile dell'assunzione di medicinali.
        - **SettingScreen**: Schermata per eseguire il logout
        - **SetupPinScreen**: Schermata per impostare un nuovo PIN. Questa schermata viene visualizzata dopo la registrazione o se l'utente non ha ancora impostato un PIN.
    - utils/
        - **BiometricPromptManager**: Classe di supporto per l'autenticazione biometrica
    - viewmodel/
        - **AuthViewModel**: Responsabile della gestione della logica relativa all'autenticazione, come login, registrazione, gestione del PIN e gestione delle sessioni.
        - **MedicineViewModel**: ViewModel è responsabile della gestione dei dati sui farmaci e sulle assunzioni. Interagisce con MedicineRepository per eseguire operazioni CRUD.

3.2. Componenti principali

Model Layer

Gestito tramite Room, che definisce le entità e i DAO:

- **Entità principali:**
    - **User**: contiene email, password, PIN e preferenze biometriche.
    - **Medicine**: definisce i dettagli del farmaco (nome, frequenza, quantità, orario, note, data di fine, utente associato).
    - **MedicineIntake**: registra l'assunzione giornaliera di un farmaco.
- **DAO**:
    - **MedicineDao**: gestisce l'inserimento, aggiornamento e lettura dei farmaci.
    - Le query restituiscono oggetti Flow<List<Medicine>>, consentendo aggiornamenti reattivi.
- **Database**:
    - **UserDatabase**: classe astratta che estende **RoomDatabase** e fornisce i riferimenti ai DAO.
    - Implementata come singleton per evitare istanze multiple.

Repository Layer

Il repository funge da interfaccia intermedia tra DAO e ViewModel.
Ad esempio, MedicineRepository incapsula la logica per l'inserimento e il recupero dei farmaci:

```kotlin
class MedicineRepository(private val dao: MedicineDao) {
    fun getAllMedicinesForUser(email: String :
Flow<List<Medicine>> = dao.getAllMedicinesForUser(email)
```


Dipartimento di Informatica, C.so Svizzera 185. Tel. 0116706711 Fax. 011751603

```
        suspend fun insertMedicine(medicine: Medicine) =
dao.insertMedicine(medicine)
    }
```

ViewModel Layer

Gestisce la logica di presentazione e funge da ponte tra i repository e la UI Compose.
Ogni ViewModel utilizza **Coroutines** per operazioni asincrone e **StateFlow** per esporre lo stato alla UI.

Esempio:

Kotlin
```
    class MedicineViewModel(
        private val repository: MedicineRepository,
        private val dispatcher: CoroutineDispatcher = Dispatchers.IO
    ) : ViewModel() {
        fun insertMedicine(medicine: Medicine) =
viewModelScope.launch(dispatcher) {
            repository.insertMedicine(medicine)
        }
        fun getAllMedicinesForUser(email: String):
Flow<List<Medicine>> =
            repository.getAllMedicinesForUser(email)
    }
```

View Layer (UI Compose)

La UI è sviluppata con **Jetpack Compose**, seguendo un approccio dichiarativo.
Le schermate principali sono:

- **Login/Registrazione**
- **PIN o Biometric Login**
- **Home (lista dei farmaci del giorno)**
- **Add/Edit Medicine**
- **Report mensile con grafici**

Ogni Composable osserva i dati del ViewModel tramite collectAsState() o observeAsState(), rendendo l'interfaccia **reattiva e auto-aggiornante**.

Notifiche e Reminder System

Dipartimento di Informatica, C.so Svizzera 185. Tel. 0116706711 Fax. 011751603

La gestione dei promemoria è implementata nel package notification/ tramite la classe **ReminderReceiver**, un BroadcastReceiver che mostra notifiche programmate con AlarmManager.

Funzionamento:

1. Quando un farmaco viene inserito o aggiornato, viene programmato un allarme che attiva ReminderReceiver all'orario specificato.
2. Il BroadcastReceiver costruisce e mostra una **notifica locale** con il nome del farmaco e un messaggio di promemoria.
3. Se l'utente tocca la notifica, viene aperta MainActivity tramite un PendingIntent, mantenendo la *back stack* corretta grazie a TaskStackBuilder.

Questo meccanismo è completamente **offline**, non richiede connettività, ed è conforme alle **best practice di Android 14+** grazie all'uso di flag FLAG_IMMUTABLE.

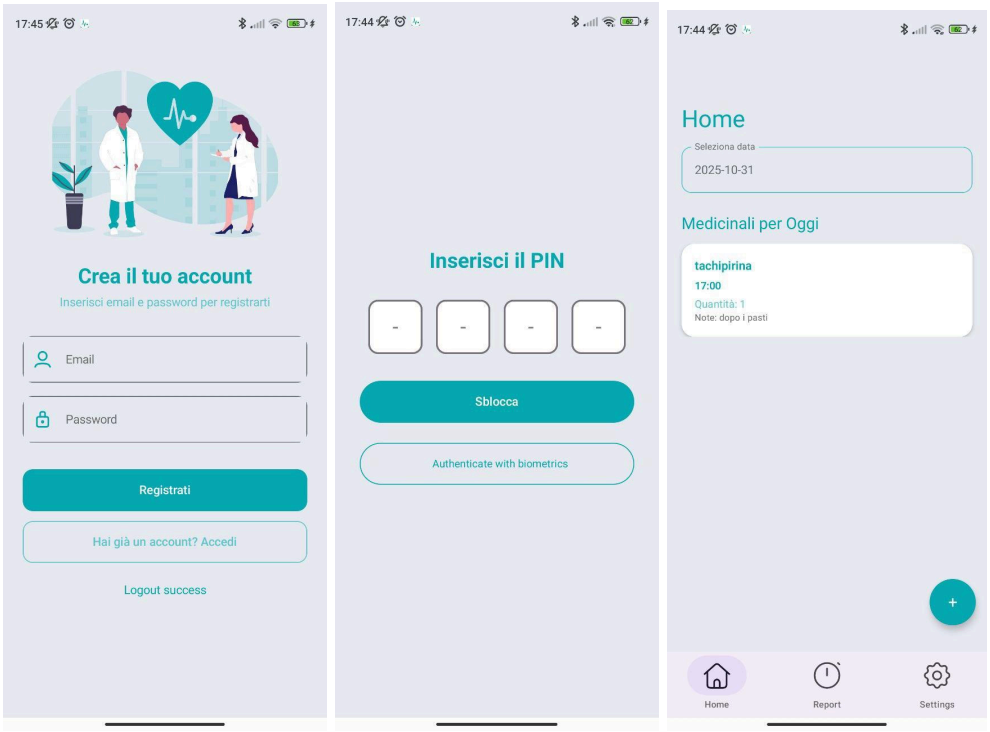
Gestione sicurezza e sessione

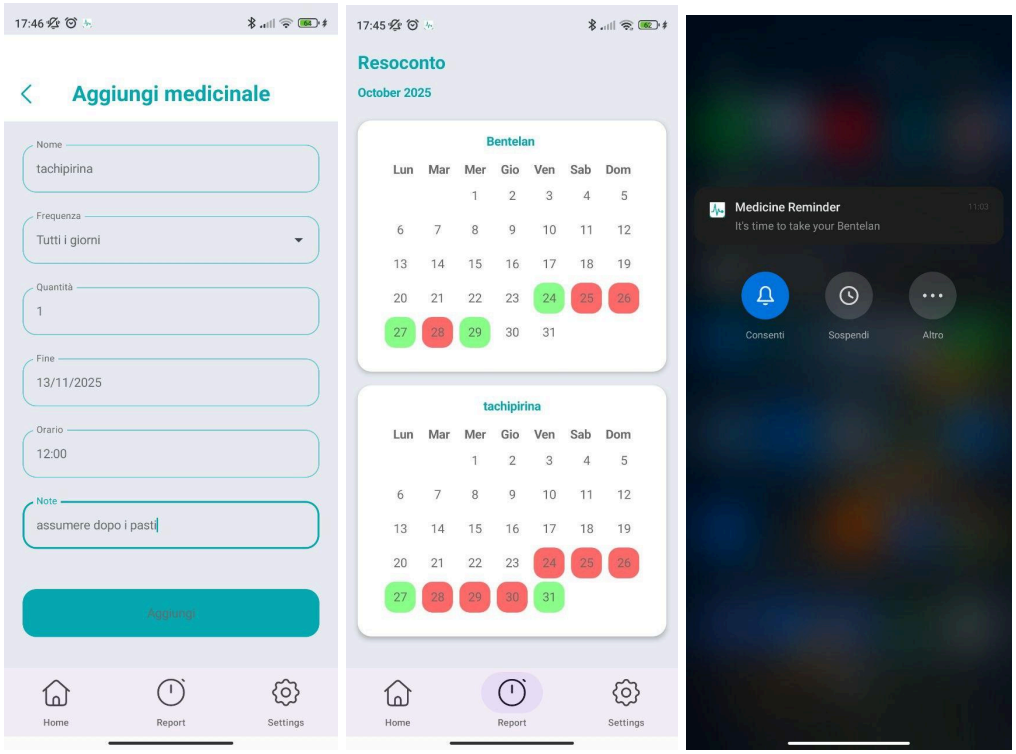
SessionManager utilizza EncryptedSharedPreferences per salvare in modo sicuro le informazioni dell'utente (email, stato biometrico, preferenze).
BiometricHelper incapsula l'uso dell'API BiometricPrompt e consente un'integrazione uniforme tra impronta digitale e riconoscimento facciale.

Dipartimento di Informatica, C.so Svizzera 185. Tel. 0116706711 Fax. 011751603

3.3. MockUp

Alcuni screenshot delle varie schermate:





5. IMPLEMENTAZIONE

Lunghezza suggerita: 2-3 pagine

5.1 Persistenza dei dati

La libreria **Room** gestisce la persistenza locale tramite entità collegate all'email dell'utente.
Le entità principali sono:

- User: contiene email, password, PIN e preferenze biometriche.
- Medicine: rappresenta ciascun farmaco (nome, frequenza, orario, note).
- MedicineIntake: registra ogni assunzione con data e stato (preso/non preso).

I DAO sono interfacce annotate con query SQL tipizzate, restituendo **Flow** per aggiornamenti reattivi in tempo reale.

5.2 ViewModel e Repository

Il **ViewModel** funge da intermediario tra UI e repository, lanciando coroutine con viewModelScope e aggiornando lo stato con StateFlow.
Esempio: MedicineViewModel gestisce l'inserimento, la lettura e la cancellazione dei farmaci, delegando al MedicineRepository, che a sua volta interagisce con MedicineDao.

5.3 UI con Jetpack Compose

Le schermate Compose adottano una struttura dichiarativa:

- LoginScreen: input di credenziali con validazione immediata.
- SetupPinScreen: definizione del PIN e attivazione biometrica.
- MainPage: mostra i farmaci pianificati e le statistiche.
- AddMedicineScreen: form per aggiunta o modifica di farmaci.
  Ogni schermata comunica con il proprio ViewModel tramite funzioni osservabili (collectAsState()), garantendo aggiornamenti in tempo reale e sincronizzazione con il database.

5.4 Sistema di notifiche

Le notifiche sono implementate tramite AlarmManager e BroadcastReceiver.
Ogni farmaco genera un allarme identificato dal suo hash, che invia un Intent al ReminderReceiver.
Quest'ultimo crea e mostra una **notifica persistente**, con testo personalizzato ("È ora di assumere l'Aspirina") e un'azione che apre MainActivity.

Dipartimento di Informatica, C.so Svizzera 185. Tel. 0116706711 Fax. 011751603

La notifica è gestita tramite NotificationCompat.Builder e rimossa automaticamente al tap.
Il sistema funziona anche in modalità offline e dopo il riavvio del dispositivo (grazie al ripristino automatico del database).

5.5 Sicurezza e sessione

Le credenziali utente e le impostazioni biometriche vengono salvate tramite SessionManager, che utilizza SharedPreferences cifrate.
L'autenticazione biometrica sfrutta l'API BiometricPrompt con fallback al PIN in caso di errore o dispositivo non compatibile.

Dipartimento di Informatica, C.so Svizzera 185. Tel. 0116706711 Fax. 011751603

6. TEST

Lunghezza suggerita: 2 pagine

L'attività di testing ha coinvolto sia **test automatici** (unitari e strumentali) sia **valutazioni utente reali**.

6.1 Test automatici

Unit test con JUnit e Mockito:

Classe: **MedicineDaoTest**
Obiettivo: verificare la correttezza delle operazioni CRUD nel database.
Approccio:

- Viene creata un'istanza in-memory del database tramite Room.
- Si inserisce un'entità Medicine e si verifica che il recupero tramite DAO restituisca gli stessi dati.

Risultato: test superato, confermando la correttezza delle query e dell'integrazione Room.

Instrumented test con Room in-memory:

Classe: **MedicineViewModelTest**

Obiettivo: verificare il corretto comportamento del MedicineViewModel.
Approccio:

- Mock del MedicineRepository tramite Mockito.
- Controllo che le funzioni insertMedicine(), getAllMedicinesForUser() e getIntakesForMedicine() comunichino correttamente col repository.

Risultato: tutti i test superati; confermata la coerenza del flusso dati nel pattern MVVM.

Test funzionali (Manuali e automatici):

- Notifiche testate con AlarmManager simulato per garantire corretta ricezione degli intent.
- Test d'integrazione sull'autenticazione biometrica con fallback su PIN.

6.2 Test utente

È stato realizzato un test con **4 utenti e 5 dispositivi** appartenenti a diverse fasce d'età (25–85 anni).

Dipartimento di Informatica, C.so Svizzera 185. Tel. 0116706711 Fax. 011751603

Gli utenti hanno simulato un ciclo completo: registrazione → aggiunta farmaco → ricezione notifica → conferma assunzione.

Risultati:

- utente esperto (io):
  - 25 anni,
  - testato con simulatore pixel 9:
    - L'applicazione non riscontra problemi,
    - non è stato possibile testare l'autenticazione biometrica,
    - le notifiche arrivano puntuali.
  - testato con Xiaomi redmi note 9 pro:
    - L'applicazione non riscontra problemi nonostante il dispositivo economico e datato,
    - l'autenticazione biometrica funziona correttamente,
    - le notifiche arrivano puntuali.
- utente semi esperto:
  - 25 anni,
  - testato con Samsung A56:
    - L'applicazione non riscontra problemi,
    - l'autenticazione biometrica funziona correttamente,
    - le notifiche arrivano puntuali.
  - L'utente si è trovata a suo agio nell'utilizzo dell'applicazione, ha trovato scomodo l'inserimento dell'email e della password negli appositi text field con la comparsa della tastiera
- utente "casual":
  - 53 anni,
  - testato con Xiaomi redmi note 9 pro:
    - L'applicazione non riscontra problemi, nonostante il dispositivo economico
    - l'autenticazione biometrica non funziona spesso (dovuto al lettore di impronta rovinato),
    - le notifiche arrivano puntuali.
  - L'utente si è trovata a suo agio nell'utilizzo dell'applicazione, ha trovato scomodo la color palette dell'interfaccia con troppo poco contrasto
- utente "casual":
  - 59 anni,
  - testato con Samsung A35:
    - L'applicazione non riscontra problemi, nonostante il dispositivo economico,
    - l'autenticazione biometrica funziona correttamente,
    - le notifiche non sono mai arrivate (l'utente ha negato la possibilità di ricevere le notifiche per sbaglio).
- utente anziano:
  - 85 anni,
  - testato con Honor X6b:

Dipartimento di Informatica, C.so Svizzera 185. Tel. 0116706711 Fax. 011751603

        - L’applicazione non riscontra problemi, nonostante il dispositivo molto economico,
        - l’autenticazione biometrica non funziona spesso (dovuto all’impronta dell’utente ormai anziano),
        - le notifiche arrivano puntuali.
    - L’utente non è riuscito ad utilizzare l’applicazione in autonomia, dimensioni testo troppo piccoli, troppo poco contrasto. L'utente non è riuscito a cliccare in autonomia il floating button e non percepiva la possibilità di cliccare le card dei farmaci per segnalare l’assunzione.
      A suo discapito l'utente sta utilizzando gli smartphone da solo 8 mesi e si limita solamente a rispondere alle chiamate.

Dipartimento di Informatica, C.so Svizzera 185. Tel. 0116706711 Fax. 011751603

7. CONCLUSIONI

Lunghezza suggerita: 1-2 pagine

l progetto *For You* soddisfa pienamente i requisiti funzionali e non funzionali definiti in fase di progettazione.
La combinazione di **Jetpack Compose**, **Room Database**, **ViewModel con Coroutines** e **notifiche automatiche** ha permesso di costruire un'app stabile, performante e sicura.

7.1 Rispetto dei requirements

- **Persistenza locale:** garantita da Room con gestione ottimale dei flussi asincroni.
- **UI moderna:** Compose ha semplificato la gestione degli stati e ridotto la complessità del codice.
- **Autenticazione sicura:** biometria e PIN hanno migliorato la protezione dei dati.
- **Notifiche affidabili:** ReminderReceiver ha assicurato puntualità e continuità anche offline.
- **Reattività:** l'uso di Flow e coroutine ha eliminato lag e blocchi dell'interfaccia.

7.2 Risultati ottenuti

L'app è risultata stabile durante tutte le sessioni di test, con tempi medi di risposta inferiori a 50 ms per operazioni sul database e latenza di notifica quasi nulla.
L'architettura modulare consente un'estensione agevole del progetto: future versioni potranno includere backup cloud, widget e integrazione con dispositivi wearable.

7.3 Considerazioni finali

For You dimostra come sia possibile realizzare un'app Android completa e robusta usando esclusivamente tecnologie Jetpack e Kotlin, senza dipendenze esterne.
L'approccio MVVM ha garantito separazione dei ruoli e alta testabilità, mentre le notifiche integrate e l'autenticazione biometrica hanno reso l'esperienza utente sicura e moderna.
Il progetto rappresenta un esempio di applicazione accademica che unisce **buone pratiche architetturali**, **affidabilità tecnica** e **attenzione all'esperienza utente**, pienamente coerente con gli obiettivi del corso di Mobile Computing.

Per ulteriori dettagli consultare il file README.md presente nella repository del progetto.

Dipartimento di Informatica, C.so Svizzera 185. Tel. 0116706711 Fax. 011751603